



QSIURP 2024

FINAL REPORT

On Improving Systems Performance Using Foundation Models

Student:
Thang Bui

Advisor:
Dr. Khaled Harras

§1 Abstract

Performant computer systems are crucial in today’s technology-driven world. Recent artificial intelligence advances, especially with generative models, present new promising directions for system optimization. In this work, we looked at a broader view of learning-based algorithms that enhance computer systems performance, with a focus on improving congestion control (CC) in computer networks. We conducted a literature survey to understand the current landscape of machine learning applications in systems tasks. We then delved deeper into the specific problem of networking systems, particularly CC, exploring the use of deep reinforcement learning (RL) to develop more efficient congestion avoidance algorithms (CAAs). Our study utilized the ns3 network simulator to lessen the difficulty of realistic data collection, addressing a critical pain point of any learning-based approach. Our evaluation demonstrated that our RL-based CAA achieved marginally better throughput compared to the widely-used TCP Cubic, highlighting the potential of using NS3 and deep RL in this domain, despite the simplicity and limited-scope of our experiment. Future work may expand on feature engineering, multi-objective optimization, and exploring new machine learning approaches, such as transformers, to further enhance adaptive congestion control algorithms.

§2 Introduction

Computer systems are the backbone that drive technological development today. Given the huge impact of computer systems, maximizing their performance is a big question that attracts a lot of attention. As computers heavily assist our modern society in many daily tasks, for instance, communication (emails), work (online conferencing), entertainment (streaming, video games), etc., performant systems benefit in many aspects. Most direct to users, better performance provides better experience and satisfaction. To businesses, optimized performance enhances reliability and scalability, allowing systems to handle growing workloads without disruptions. Furthermore, efficient systems performance reduces operational costs by maximizing resource utilization while minimizing energy consumption. Overall, achieving better performance is an active computer science research area [1]–[3].

Needless to say, designing algorithms that maximize performance is difficult. Traditional rule-based solutions for systems tasks tend to fail short in some aspects. First, computer systems are expected to work under various environments, for example, an operating system runs on top of different hardware and user workload. Each environment has unique characteristics, which may be handled well by a generic solution. Second, in most cases, computer systems cannot fully observe the input. To illustrate, in the process scheduling task above, we cannot fully know the application behavior, i.e. memory usage, data fetching pattern, etc., all of which can affect the execution time. Not knowing the full picture impedes the ability to make the most rational decisions. Lastly, actions for one task may impact the status of others because a system can be responsible for multiple tasks, for instance, an operating system consists of process scheduling, cache management, etc., and each decision for one task can influence the performance of others. Failure to account for holistic dynamics can also reduce systems performance. These complications, namely, inadaptability, partial observability, and lack of consideration of task synergy, render algorithms that are often suboptimal [1].

Inspired by recent artificial intelligence (AI) advances, in this project, we aim to explore AI applications to computer system tasks. The capabilities of foundation models

(FMs) and large language models (LLMs) in solving sequence-to-sequence problems in other domains such as natural language processing or computer vision has spurred the discussion if they can help designing better algorithms for system tasks because many system tasks may also be modeled as sequence-to-sequence. The remaining of our report is structured as follows: section 3 surveys the existing literature of ML for systems and other crucial concepts in our work; section 4 details our work in improving congestion control in computer networks using deep reinforcement learning; section 5 and 6 discuss some preliminary results and potential future works.

§3 Background

In this section, we provide an overview of the relevant work and concepts essential to our project. Section 3.1 summarizes the current landscape of machine learning (ML) applications in systems that lays out available research paths. Subsequently, section 3.2 dives deeper into ML for congestion control, driven by personal interest, to explore previous works in greater detail. Lastly, section 3.3 presents a brief overview of reinforcement learning, which forms the foundation for our research direction in the following sections.

§3.1 Machine Learning for Systems

Since the rise of ML in recent decades, there has been much research on its applications to improve computer system tasks, or ML for systems. Existing works have explored the use of a diverse set of ML techniques in various systems, including cloud systems, databases, computer networks, etc. [2], [3]. Figure 1 provide a taxonomy of existing ML for system work. Table 3.1 breaks down about what each task concerns, the limitations of traditional heuristics that motivate ML application research, and existing ML approaches for each task from prior works. These summaries provide us an initial overview of available directions for deeper exploration.

§3.2 Machine Learning for Congestion Control

Congestion control (CC) aims to optimize the utilization of network resources by regulating data flow within the network. Computers communicate by transmitting data through intermediate devices and links, each with its own constraints: bandwidth (data capacity), error rate (data integrity during transmission), and propagation delay (time for data to travel to the next point). Just as traffic systems can become overwhelmed with too many vehicles, networks can collapse under excessive data flow, leading to undesirable scenarios, e.g., degraded performance, packet loss, increased latency, and underutilized resources.

Congestion avoidance algorithms (CAAs) form the foundation of CC. A network connection typically involves a sender and a receiver. CAAs prevent congestion by adjusting the sender’s data rate based on observable metrics such as delay and loss. For instance, the algorithm may reduce the congestion window (the volume of data sent simultaneously) in response to high loss rates and increase it when conditions improve to maximize resource use and performance. Beyond congestion avoidance and performance, CAAs may also consider other objectives, such as fairness (equitable resource sharing among multiple connections) and friendliness (compatibility with other CAAs).

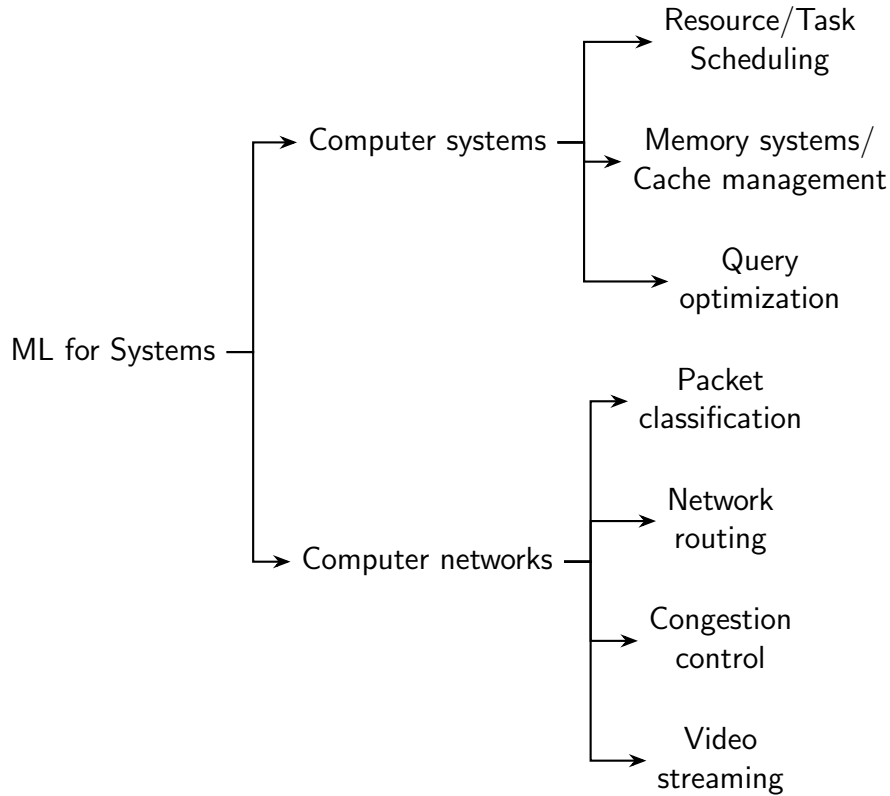


Figure 1: Taxonomy of ML applications in system tasks

Research in congestion control has a long history, closely tied to the development of the reliable Transmission Control Protocol (TCP), which is extensively used by numerous Internet applications today. Over the years, many TCP variants, e.g. Tahoe, Reno, Vegas, etc., have been developed. TCP Cubic is arguably the most notable variant and has been the default CC algorithm in the popular Linux kernel for nearly two decades. However, these heuristic schemes often excel in specific scenarios but fail in others. For example, Yen and Abbasloo et al. [4] showed that heuristics that perform well in single-flow scenarios (focusing on throughput and delay) are inefficient in multi-flow scenarios (requiring fair resource sharing). Designing effective CAAs remains an ongoing research challenge.

Recent advances in artificial intelligence (AI) and machine learning (ML) have spurred projects leveraging ML to improve congestion control. Many prior works aimed to create adaptive CAAs that perform well across diverse environments. PCC [5] employs online learning techniques similar to A/B testing to determine the optimal congestion window. Aurora [6] demonstrated the advantages and potentials of deep reinforcement learning (RL)-based CC over traditional heuristic methods. Orca [7] proposes a two-level approach that combines classic heuristics for fine-grained with deep RL for coarse-grained CC to reduce the inference overhead of learning-based schemes. More recent works like Antelope [8] and Sage [4] showcase new schemes that dynamically select or learn from the best heuristic decisions, offering adaptable and high-performance algorithms.

Building the necessary infrastructure for learning-based approaches is also crucial. Mahimahi [9], a network emulator, and Pantheon [10], an evaluation platform, have been widely adopted in academic CC research. NS3 [11], a well-established network simulator, is frequently used in network research, including CC studies.

I conducted an in-depth analysis of Sage [4], an offline RL framework that improves

Task	Description	Heuristic limitations	ML techniques
Resource/Task Scheduling	Provision resources and distribute tasks to meet goals on resource utilization, response time, fairness, and affinity constraints	cannot model accurately resources and tasks; some metrics is difficult to hand-craft heuristic rules	Elastic resource provision based on workload prediction; heterogeneity and interference-aware scheduling
Memory/Cache management	Modern systems contain multi-layered memory, each layer creates performance bottlenecks	Rule-based solutions are suboptimal because data pattern is too complex	Predict upcoming memory access from historical patterns; learning efficient representations of memory access patterns
Query optimization	Determining the most efficient way to execute a given query by considering (all) possible query plans	Data patterns are too complex to be modelled by general heuristics	Learned indexing; estimate query cardinality; join ordering
Packet classification	Match packets to a rule from a specified collection, e.g., source and destination IP addresses,	Fail to exploit characteristics of a given rule set	Using ML to learn classifier model or clustering; using RL agents to generate decision trees at runtime according to the given set of rules
Network routing	Determine the optimal adjacent node to forward packets	Difficult to hand-craft heuristics to optimize for multi-faceted goals	Using Q-learning to learn an efficient policy
Video streaming	Select a suitable bitrate based on the real-time network condition	Difficulty to model relationship between bandwidth estimation, buffer occupancy, and bitrate	Using ML and RL to make better estimations and decisions
Congestion control	discussed in 3.2	discussed in 3.2	discussed in 3.2

Table 1: Breakdown of ML applications in system tasks

on heuristic designs by thoroughly examining their paper and scrutinizing available code artifacts. From my literature review, I identified the following drawbacks in prior works, though I focused only on the first due to the limited timeline of QSIURP:

- Efficient data collection from diverse network environments is challenging for learned CC solutions. While Sage and others use network emulators like Mahimahi, this requires kernel-based implementations or socket option APIs, which can be significant barriers. An alternative approach is using network simulators like NS3 to generate data based on specifications, which very few works have pursued.
- Sage’s model decides the congestion window at very fine intervals (e.g., every 10 milliseconds), which may be impractical due to longer inference times for ML models. A potential solution is a two-level control system where the model selects

the underlying heuristic algorithm at longer intervals, allowing the heuristic to manage finer control.

- Sage utilizes GRUs and Gaussian mixture models as its neural network architecture. Newer approaches suggest transformers, which leverage attention mechanisms and have shown superior performance in sequence-to-sequence tasks, such as language models like ChatGPT. Exploring transformers for CC, replacing GRUs and GMMs, remains an uncharted territory.

Despite time constraints, these identified areas offer promising directions for future research in congestion control.

§3.3 Reinforcement Learning

RL is a subfield of machine learning in which an agent learns how to behave in an environment by performing actions and observing the consequences of those actions. At each discrete timestep $t = 0, 1, \dots$, the agent observes a local state s_t from the environment and acts according to a selected action a_t . At the following time step $t + 1$, the agent receives a reward r_t , representing its loss or gain after time t , in addition to the next state s_{t+1} . The central objective in RL is to find a policy π mapping states to actions that maximize the expected cumulative discounted return $R_t = \mathbb{E}[\sum_t \gamma^t r_t]$, where γ is the discount factor. Due to the computational challenges posed by large or continuous state and action spaces, deep RL emerged, leveraging neural networks to approximate the optimal π .

RL has several advantages over other machine learning paradigms. Among the most significant are its ability to learn from interaction and handle delayed rewards. RL agents learn directly from interactions with their environment, without the need for a dataset, which is essential for supervised and unsupervised learning. Additionally, RL works well in scenarios where rewards are delayed, where an immediate action can influence many subsequent states because RL agents learn to act to maximize cumulative rewards over time, even when the actions do not yield immediate benefits. In contrast, both supervised and unsupervised learning focus on immediate feedback and struggle with situations involving delayed rewards. These strengths make RL particularly well-suited for tasks requiring sequential decision-making in complex and dynamic environments, such as congestion control in computer networks.

The long history of RL research has developed various techniques for training RL agents [12]. Figure 2 presents a taxonomy of RL training algorithms. In our work, we focus on the Proximal Policy Optimization (PPO) algorithm. As a policy optimization method, PPO offers several advantages over other approaches. In model-based RL, the agent attempts to model the environment based on its interactions and then uses this learned model to make decisions. However, this approach requires a substantial amount of data to accurately model the environment, making it more data-intensive than policy optimization algorithms. Another category of RL algorithms is Q-learning, where the agent learns to approximate the Q-function, representing the total discounted reward for taking an action in a given state. Q-learning’s inherent limitations are being computationally intensive and its inability to learn stochastic policies, which can better smooth out optimization processes. In contrast, PPO strikes a strong balance between stability and performance while being relatively easy to implement compared to other policy optimization algorithms. These qualities make PPO an excellent starting point when experimenting with RL algorithms.

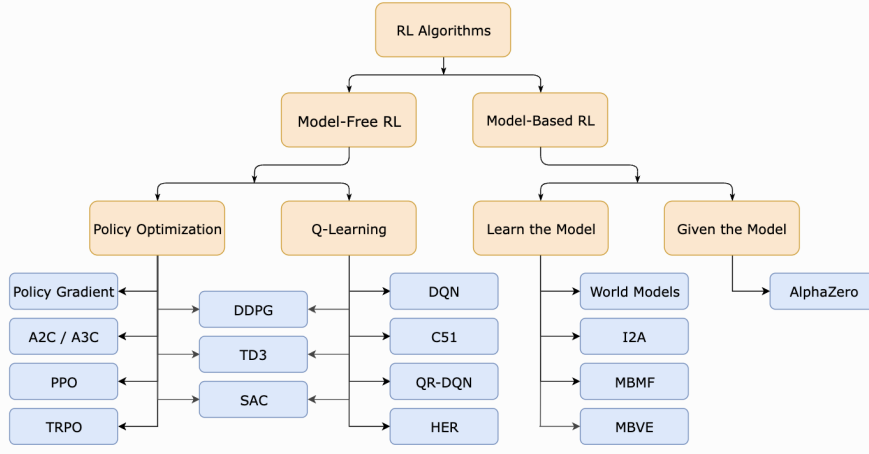


Figure 2: A taxonomy of RL training algorithms [12]

§4 Approach

In this work, we focused on overcoming the data collection challenges in learning-based congestion control (CC) schemes. Prior research often requires additional dependencies and special privileges for data collection, which can be a significant barrier for researchers. The NS3 network simulator is a well-established tool for generating realistic data without such requirements and allows fine-grained control over network parameters. However, NS3’s incompatibility with the ML ecosystem and its complex configuration can be difficult for any users. This section outlines our approach to developing an RL training platform and experimenting with a deep RL model for CC using the NS3 network simulator.

§4.1 Architecture

Bridging the gap between NS3 and the ML ecosystem requires additional effort. The NS3 network simulator is written in C++, while most ML libraries are in Python, leading to compatibility issues. Fortunately, existing solutions address this problem. ns3gym [13] is a Python framework that integrates OpenAI Gym, a core dependency for RL providing a standard API for communication between learning algorithms and environments, with NS3 to encourage RL use in networking research. Figure 3 illustrates the components in my experimental setup.

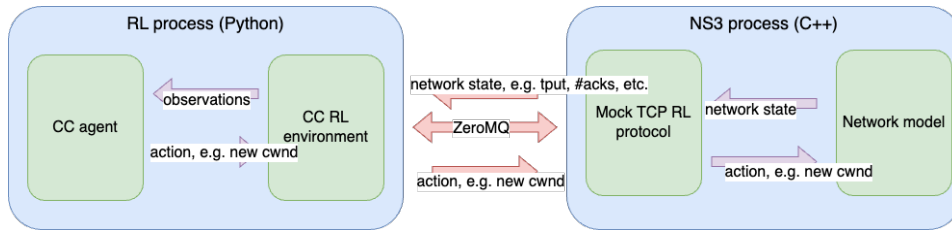


Figure 3: Training platform architecture

To integrate NS3 with Python RL libraries, ns3gym launches a child process of running Python process to run NS3. Therefore, there will be 2 processes at runtime: the parent Python process concerning RL training and the child C++ process that runs NS3 to simulate the network. The two processes communicate with each other using interprocess

communication mechanism provided by ZeroMQ library. During training, first ns3gym spawns the child NS3 process and bootstrap the network model. The network model will notice the mock TCP RL protocol for any CC event such as changes in RTT estimation. The mock TCP RL protocol accumulates these notifications and update the network state to CC RL environment every monitor interval, which in turn will be passed to the CC agent as the environment observation. The CC agent acts based on the passed observation, and the action is passed all the way back to the network model, reflecting a change to the network. The action-feedback loop continues as the RL agent learns to master CC.

§4.2 RL environment

I formulated CC as a sequential decision-making problem within the RL framework:

Monitor Intervals (MIs): We adopted the notion of monitor intervals by Dong et al. [5]. Continuous time is divided into discrete monitor intervals. The duration of these intervals is a hyperparameter. The intervals are 0.5 seconds in our experiment allowing sufficient time for agents’ actions to take effect.

Agents and Actions: The agent is the data sender. At the beginning of each monitor interval, the agent selects the congestion window size as its action. I chose to set the congestion window size based on the number of packets rather than bytes and limited the action space to 1-1000 packets. Thus, the congestion window in bytes is the product of the number of packets and packet size. This allows quick adjustments across different magnitudes.

Avoiding Congestion Window Explosion: I decided against setting actions as changing ratios, as other works do, to prevent the congestion window from rapidly increasing to a huge number. Clipping the congestion window size in this case does not effectively differentiate between large sizes.

Observations: After selecting the congestion window $cwnd_t$ at MI_t , the sender observes the effects of sending at that rate and computes an observation vector obs_t from received packet acknowledgements at the end of MI_t . For simplicity, I included following common CC-relevant features in obs_t :

- current congestion window size $cwnd_t$
- round-trip time (RTT) as estimated by ns3
- loss rate: $1 -$ (the ratio of total packets acknowledged over total packets sent)
- throughput: the ratio of packets acknowledged in MI_t over the duration of MI_t

Network topology: We experimented in a single-flow scenario with 1 sender and 1 receiver over a dumbbell topology as illustrated in Figure 4 [14]. The bandwidth and one-way delay of the bottleneck link varied to facilitate model learning and adaptation. When the environment starts a new RL episode, it randomly selects bandwidth and delay values within a specified range, and the RL agent begins sampling. Table 2 listed the values of several network parameters.

Training: We employed Proximal Policy Optimization (PPO) [15] implementation in Stable Baselines3 library [16] with a multi-layer perceptron (MLP) architecture for policy training, as it has proven effective in various tasks, including CC, in previous works. Both the policy and critic networks in our experiment are MLPs with two hidden layers, each containing 128 neurons.

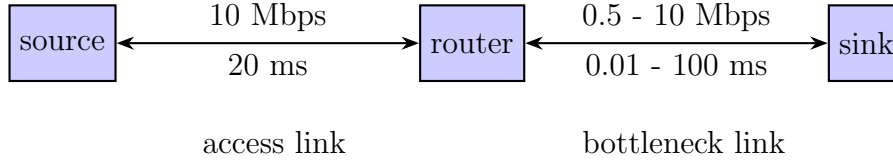


Figure 4: Training network topology

Parameter	Value
Access link bandwidth	10Mbps
Access link one-way delay	20ms
Bottleneck link bandwidth	0.5Mbps - 10Mbps
Bottleneck link one-way delay	0.01ms - 100ms
MTU	1500 bytes
Error rate	0.0
Application type	Bulk send application
Sampling episode duration	60 seconds

Table 2: Experimental network parameters

Evaluation: For evaluation, we compared our policy’s performance against TCP Cubic using the similar single-flow dumbbell scenario during training but with the fixed bottleneck link that has bandwidth 10Mbps and delay 0.01ms.

§5 Preliminary Result and Discussion

The experiment aimed to investigate how the RL model behaves when tasked with maximizing network throughput. The reward function at each MI was calculated as the number of bytes the receiver gets during that MI, divided by the MI duration (0.5 seconds). We set the learning rate to constant value $3e-4$, a commonly used value. The expected behavior of the RL model was to maintain a large congestion window to maximize throughput. Figure 5 demonstrates our training result.

The reward plot shows rapid initial learning until around timestep 150000, after which the episodic reward fluctuates significantly, suggesting the model has stopped learning. This may be due to the constant learning rate, preventing the model from making smaller steps to reach optimal performance.

Nevertheless, other plots confirm the hypothesized behavior of the RL agent. The congestion window plot over time clearly shows the RL model quickly increases the sending rate and maintains it at a high level compared to TCP Cubic. The high RTT indicates the router queue is always full, explaining the overall high loss rate due to packet drops from the full queue. This behavior is justified as the reward function does not penalize high loss rates. Despite this, the total acknowledgement plot over time indicates that the model achieves slightly better results than TCP Cubic in the evaluation setting.

§6 Conclusion and Future Work

In this project, we reviewed existing ML applications in system works with special focus on ML for congestion control. We utilized NS3 network simulator to lower the access

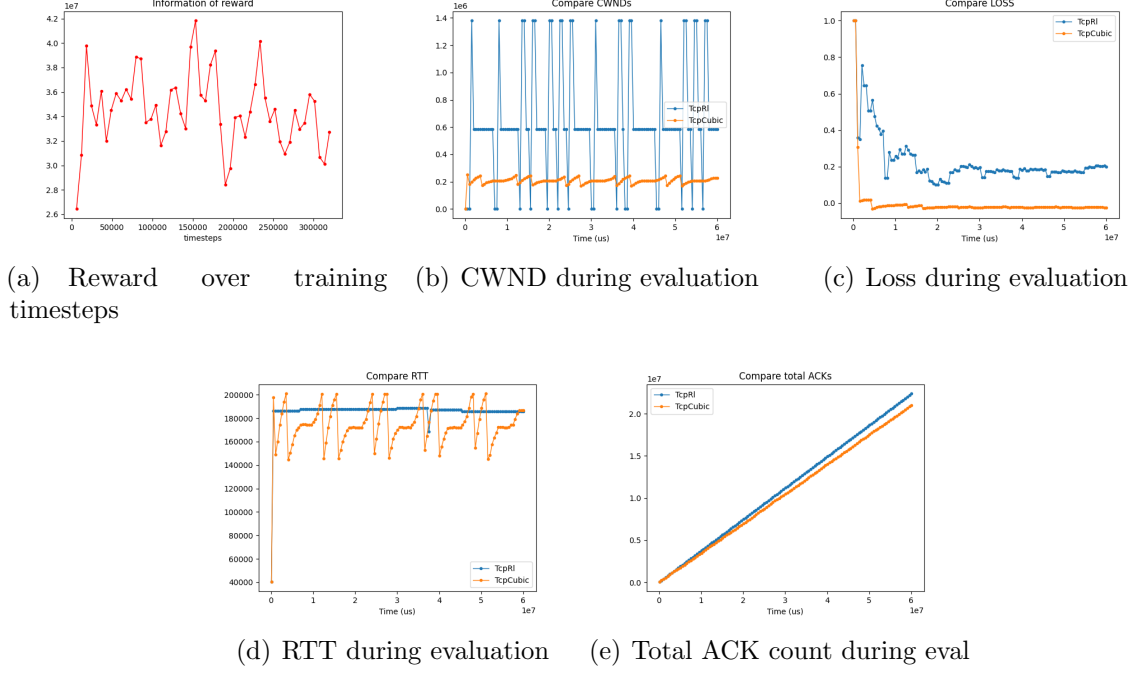


Figure 5: Training and evaluation results

barrier to realistic data for ML-based congestion control research, which is a direction that has not been extensively studied. Our simple experiment delivered a learning-based CC algorithm that slightly outperformed TCP Cubic, a widely used heuristic scheme, in terms of throughput, despite its ignorance of other performance metrics. This underscores the potential of using the ns3 network simulator to generate data for more extensive congestion control research, particularly in developing adaptive CC algorithms.

Future research could build on these results by delving into the following areas:

- **Feature Engineering**: Our experimental setup included only three basic network features related to congestion control — throughput, loss, and delay — as inputs to our model. Future studies can engineer more explicit features that better represent congestion events, such as timeouts, or incorporate statistics from previous time steps to provide the RL model with clearer network states. More detailed features would enable the RL model to better differentiate network states and make more informed decisions.
- **Multi-Objective Policy Optimization**: Our reward function focused solely on optimizing throughput, ignoring other important metrics such as loss, delay, fairness, and friendliness. This narrow focus can prevent the policy from achieving optimal performance. Future research should conduct more comprehensive experiments to refine reward functions for policies that consider multiple objectives.
- **Alternative Approaches for Adaptive CC Algorithms**: A challenge for several learning-based approaches is the long inference time, which some prior works have attempted to mitigate using a two-level control. Additionally, the emergence of transformers and foundation models presents new directions for ML applications in congestion control. These promising areas were not explored in our study due to time constraints but warrant further investigation in future work.

§7 Bibliography

- [1] D. Saxena, N. Sharma, D. Kim, *et al.*, “On a foundation model for operating systems,” 2023. arXiv: [2312.07813](https://arxiv.org/abs/2312.07813) [cs.OS].
- [2] N. Wu and Y. Xie, “A survey of machine learning for computer architecture and systems,” *ACM Comput. Surv.*, vol. 55, no. 3, Feb. 2022, ISSN: 0360-0300. DOI: [10.1145/3494523](https://doi.org/10.1145/3494523). [Online]. Available: <https://doi.org/10.1145/3494523>.
- [3] M. E. Kanakis, R. Khalili, and L. Wang, “Machine learning for computer systems and networking: A survey,” *ACM Comput. Surv.*, vol. 55, no. 4, Nov. 2022, ISSN: 0360-0300. DOI: [10.1145/3523057](https://doi.org/10.1145/3523057). [Online]. Available: <https://doi.org/10.1145/3523057>.
- [4] C.-Y. Yen, S. Abbasloo, and H. J. Chao, “Computers can learn from the heuristic designs and master internet congestion control,” in *Proceedings of the ACM SIGCOMM 2023 Conference*, ser. ACM SIGCOMM ’23, New York, NY, USA: Association for Computing Machinery, 2023, pp. 255–274, ISBN: 9798400702365. DOI: [10.1145/3603269.3604838](https://doi.org/10.1145/3603269.3604838). [Online]. Available: <https://doi.org/10.1145/3603269.3604838>.
- [5] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, “PCC: Re-architecting congestion control for consistent high performance,” in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, Oakland, CA: USENIX Association, May 2015, pp. 395–408, ISBN: 978-1-931971-218. [Online]. Available: <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/dong>.
- [6] N. Jay, N. Rotman, B. Godfrey, M. Schapira, and A. Tamar, “A deep reinforcement learning perspective on internet congestion control,” in *Proceedings of the 36th International Conference on Machine Learning*, K. Chaudhuri and R. Salakhutdinov, Eds., ser. Proceedings of Machine Learning Research, vol. 97, PMLR, Sep. 2019, pp. 3050–3059. [Online]. Available: <https://proceedings.mlr.press/v97/jay19a.html>.
- [7] S. Abbasloo, C.-Y. Yen, and H. J. Chao, “Classic meets modern: A pragmatic learning-based congestion control for the internet,” in *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, ser. SIGCOMM ’20, Virtual Event, USA: Association for Computing Machinery, 2020, pp. 632–647, ISBN: 9781450379557. DOI: [10.1145/3387514.3405892](https://doi.org/10.1145/3387514.3405892). [Online]. Available: <https://doi.org/10.1145/3387514.3405892>.
- [8] J. Zhou, X. Qiu, Z. Li, *et al.*, “A machine learning-based framework for dynamic selection of congestion control algorithms,” *IEEE/ACM Transactions on Networking*, vol. 31, no. 4, pp. 1566–1581, 2023. DOI: [10.1109/TNET.2022.3220225](https://doi.org/10.1109/TNET.2022.3220225).
- [9] R. Netravali, A. Sivaraman, S. Das, *et al.*, “Mahimahi: Accurate record-and-replay for http,” in *Proceedings of the 2015 USENIX Conference on Usenix Annual Technical Conference*, ser. USENIX ATC ’15, Santa Clara, CA: USENIX Association, 2015, pp. 417–429, ISBN: 9781931971225.

- [10] F. Y. Yan, J. Ma, G. D. Hill, *et al.*, “Pantheon: The training ground for internet congestion-control research,” in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, Boston, MA: USENIX Association, Jul. 2018, pp. 731–743, ISBN: 978-1-939133-01-4. [Online]. Available: <https://www.usenix.org/conference/atc18/presentation/yan-francis>.
- [11] G. F. Riley and T. R. Henderson, “The ns-3 network simulator,” in *Modeling and Tools for Network Simulation*, K. Wehrle, M. Güneş, and J. Gross, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 15–34, ISBN: 978-3-642-12331-3. DOI: [10.1007/978-3-642-12331-3_2](https://doi.org/10.1007/978-3-642-12331-3_2). [Online]. Available: https://doi.org/10.1007/978-3-642-12331-3_2.
- [12] OpenAI, *Kinds of rl algorithms*. [Online]. Available: https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html.
- [13] P. Gawłowicz and A. Zubow, “ns-3 meets OpenAI Gym: The Playground for Machine Learning in Networking Research,” in *ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*, Miami Beach, USA, Nov. 2019. [Online]. Available: http://www.tkn.tu-berlin.de/fileadmin/fg112/Papers/2019/gawlowicz19_mswim.pdf.
- [14] S. Gangadhar, T. A. N. Nguyen, G. Umapathi, and J. P. G. Sterbenz, “Tcp westwood(+) protocol implementation in ns-3,” in *Proceedings of the 6th International ICST Conference on Simulation Tools and Techniques*, ser. SimuTools ’13, Cannes, France: ICST (Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering), 2013, pp. 167–175, ISBN: 9781450324649.
- [15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal policy optimization algorithms*, 2017. arXiv: [1707.06347](https://arxiv.org/abs/1707.06347) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1707.06347>.
- [16] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>.